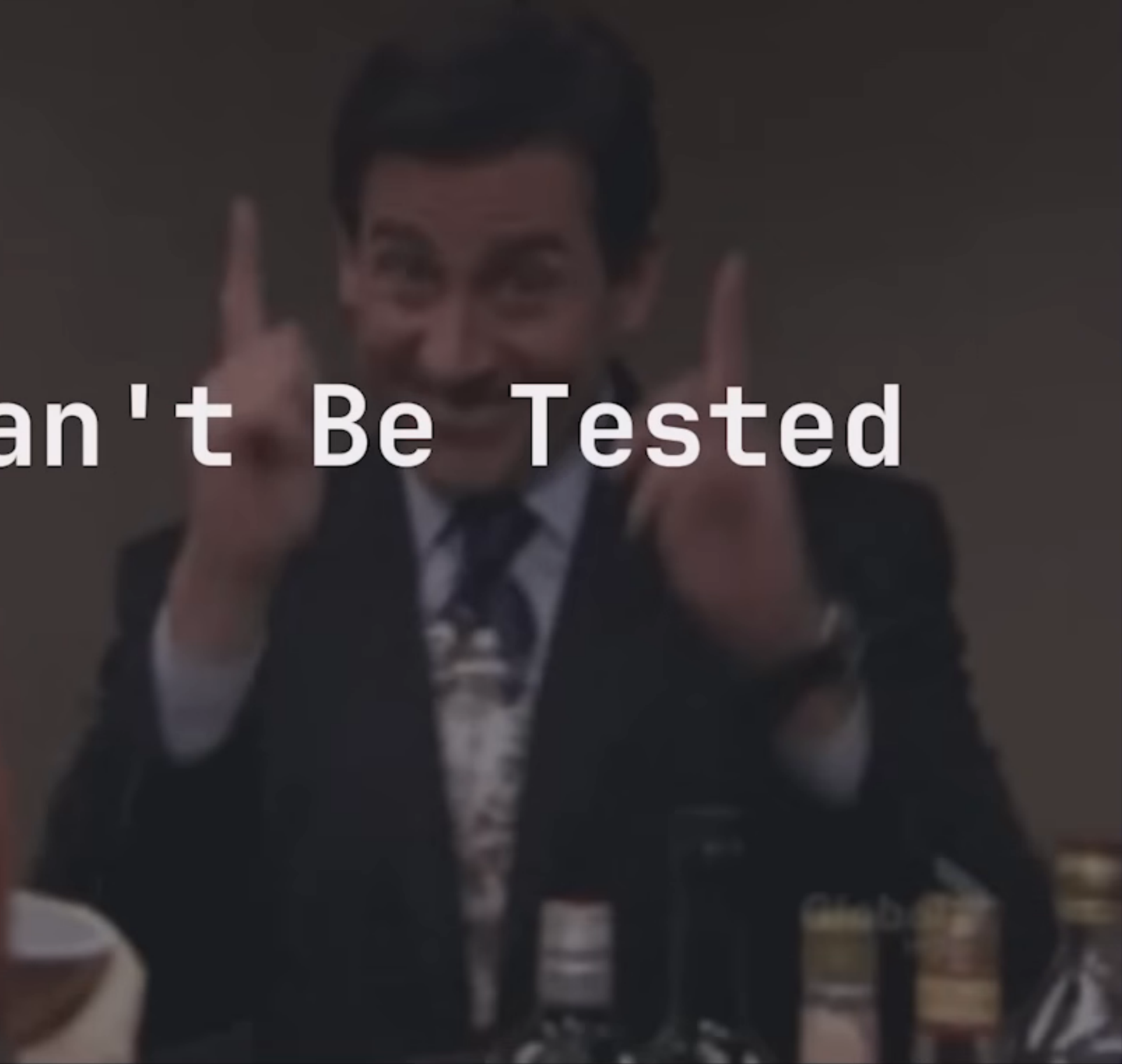


"This Can't Be Tested"

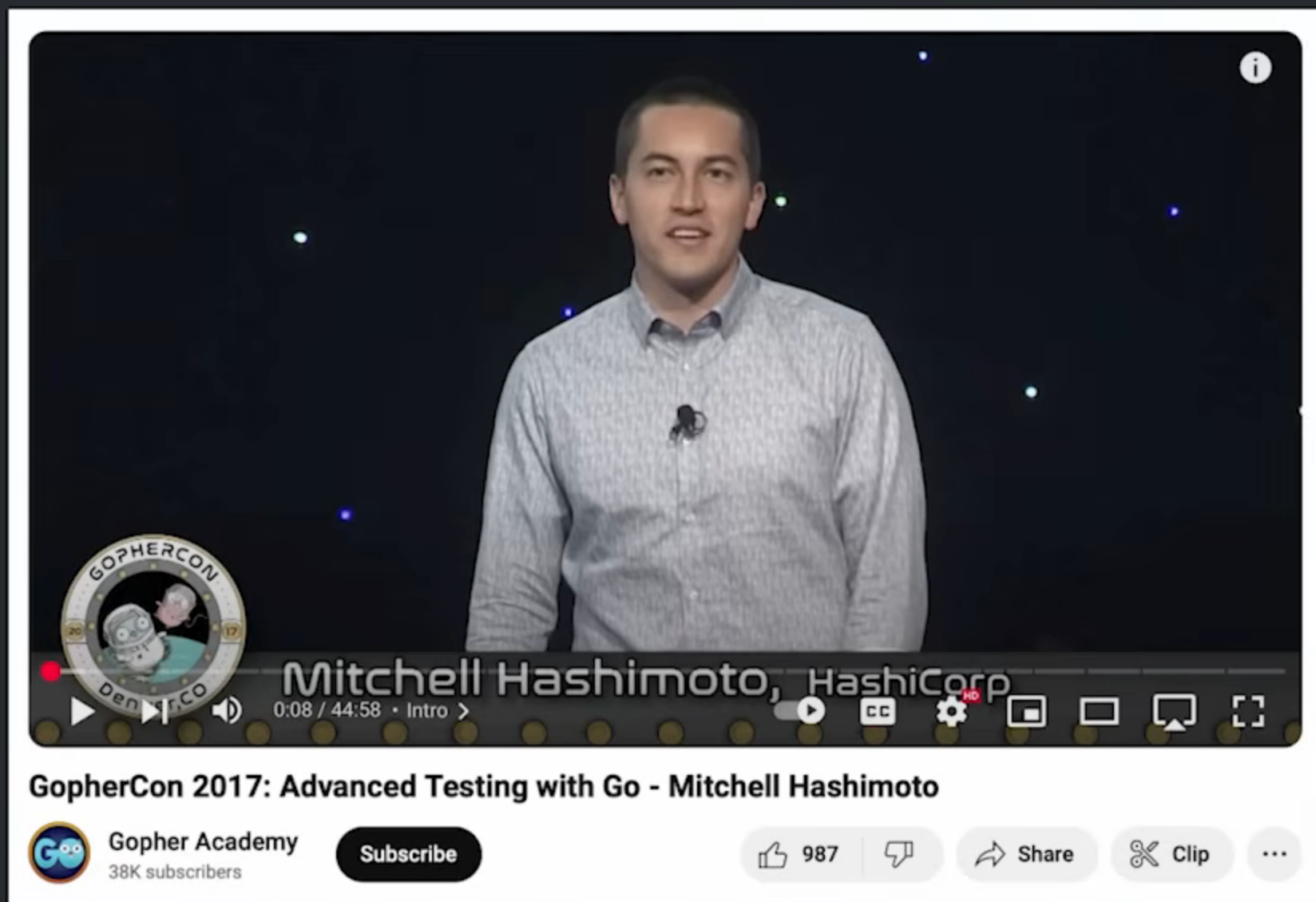
"This Can't Be Tested
Yet"



My Background

- ~12y HashiCorp
 - Networked systems
 - Large scale
 - Security-sensitive
- ~3y Ghostty
 - Desktop software
 - macOS and Linux
 - GPU rendered

2017: "Advanced Testing with Go"



What We're Taught

```
expect (add (1, 2) == 3) ;
```

What We're Taught

```
expect (add(1, 2) == 3);
```

Reality:

`add` is executing `echo "1 + 2" | bc`
and our software claims to be
compatible with Windows. So nothing
works and everything is terrible.¹

1. This is a silly, exaggerated example, of course!

But *Reality is Messy*

- Side effects
- State setup
- Networking
- GPUs
- Concurrency
- Unclear pre/post-conditions
- Complex outputs (e.g. images)
- High-dimensionality
- etc...

Two Sides to Testing:



Strategy

How to test something



Testability

How to make testable software

- Code structure
- Available APIs, config, etc.
- Packaging
- Automation harnesses



Snapshot Testing¹

Verify produced output against some ground truth

- Scenario: Complex output formats, significant quantity of comparisons, improved diff on failure
- Don't forget to provide an easy way to update the ground truth when needed
- Leverage VCS for reviewing test changes

1. née "Golden Files"

Snapshot Testing

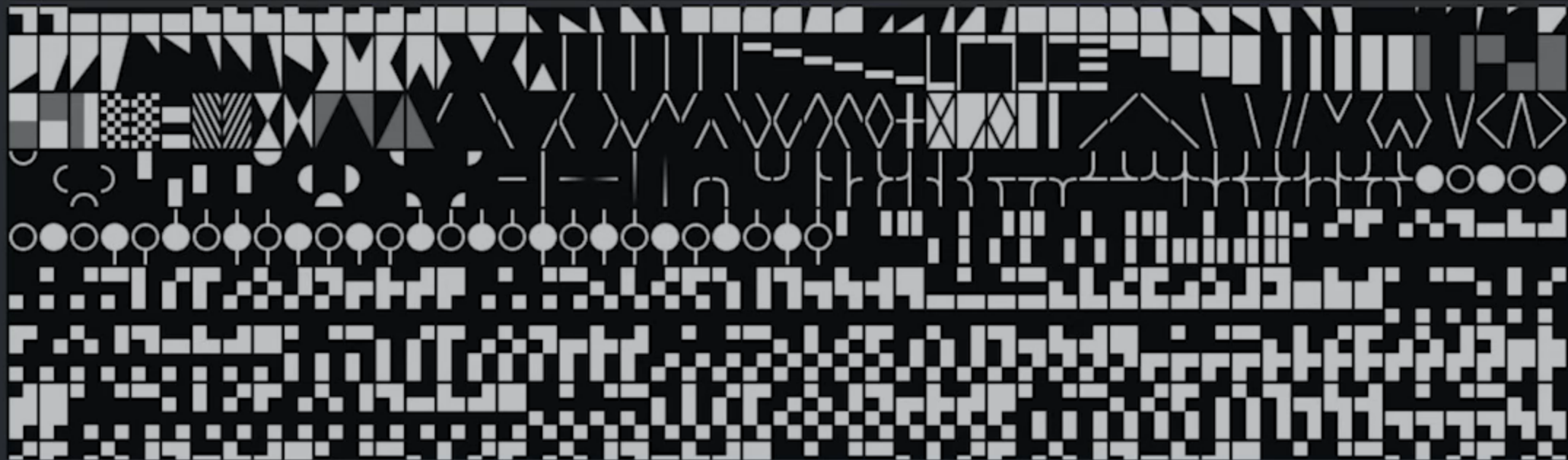
```
test "render all sprites" {
  var atlas: font.Atlas = try .init(alloc, .grayscale);
  defer atlas.deinit(alloc);

  // Even cell size and thickness (18 x 36)
  try (Box{
    .metrics = .calc(.{
      .cell_width = 18.0,
    }),
  }).testRenderAll(alloc, &atlas);

  // Odd cell size and thickness (9 x 15)
  try (Box{
    .metrics = .calc(.{
      .cell_width = 9.0,
    }),
  }).testRenderAll(alloc, &atlas);

  const ground_truth = @embedFile("./testdata/Box.ppm");
  var stream = std.io.changeDetectionStream(ground_truth);
  try atlas.dump(stream.writer());

  try testing.expect(!stream.changeDetected());
}
```





Snapshot Testing

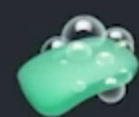
Bonus: Some ground truth formats enable more helpful test failure information





Keep Context Close

```
test "Terminal: eraseDisplay below protected attributes respected with iso mode" {  
  var t = try init(alloc, .{ .rows = 5, .cols = 5 });  
  defer t.deinit(alloc);  
  
  t.setProtectedMode(.iso);  
  for ("ABC") |c| try t.print(c);  
  t.carriageReturn();  
  try t.linefeed();  
  for ("DEF") |c| try t.print(c);  
  t.carriageReturn();  
  try t.linefeed();  
  for ("GHI") |c| try t.print(c);  
  t.setCursorPos(2, 2);  
  t.eraseDisplay(.below, false);  
  
  {  
    const str = try t.plainString(testing.allocator);  
    defer testing.allocator.free(str);  
    try testing.expectEqualStrings("ABC\nDEF\nGHI", str);  
  }  
}
```

Isolate Side-Effects

Extract purely functional behaviors

- Scenario: Behavior intermixed with IO or complex setup requirements
- Classic examples: IO event handlers such as connections, mouse click, keyboard input, etc.
- Garbage In, Garbage Out
 - But we can test the garbage



Isolate Side-Effects

Before

getMouseState()
checkMouseState()
getKeyboardState()
checkKeyboardState()
readSettings()
encodeKey()
writeToPty()

After

getMouseState()
getKeyboardState()
readSettings()
KeyEncoder
writeToPty()

Isolate Side-Effects

```
event: KeyEvent,
macos_option_as_alt: config.OptionAsAlt = .false,
alt_esc_prefix: bool = false,
cursor_key_application: bool = false,
keypad_key_application: bool = false,
ignore_keypad_with_numlock: bool = false,
modify_other_keys_state_2: bool = false,
kitty_flags: KittyFlags = .{},

const KeyEvent = struct {
    action: Action = .press,
    key: Key,
    physical_key: Key = .invalid,
    mods: Mods = .{},
    consumed_mods: Mods = .{},
    composing: bool = false,
    utf8: []const u8 = "",
    unshifted_codepoint: u21 = 0,
};

const KittyFlags = packed struct(u5) {
    disambiguate: bool = false,
    report_events: bool = false,
    report_alternates: bool = false,
    report_all: bool = false,
    report_associated: bool = false,
};

test "kitty: report alternates with ru layout" {
    var enc: KeyEncoder = .{
        .event = .{
            .key = .semicolon,
            .mods = .{},
            .utf8 = "q",
            .unshifted_codepoint = 1095,
        },
        .kitty_flags = .{
            .disambiguate = true,
            .report_all = true,
            .report_alternates = true,
            .report_associated = true,
        },
    };

    var buf: [128]u8 = undefined;
    const actual = try enc.kitty(&buf);
    try testing.expectEqualStrings("\x1b[1095::59;;1095u", actual);
}
```



GPU



GPU Testing

Treat the GPU pipeline as a function

- Scenario: Any type of shader pipeline
- Two sides:
 - CPU emits correct buffer data
 - GPU emits correct buffer data
- GPU simplification: $f(x) = y$
 - y comes out on a buffer (texture)
- Plays well with snapshot testing, but not required



GPU Testing

CPU side:

```
f(state) = (V, E, Data)
```

```
assert(f(state) == expected);
```

Given world state, the CPU produces a graph of steps (i.e. vertex, fragment, compute) with attached data (literal byte buffers)

- Isolate the Side-Effects! 🚗



GPU Testing

GPU side:

```
f(X) = Y  
X, Y = set of buffers
```

General idea:

1. Artificially construct input buffers
2. Ensure output buffers are CPU-accessible
3. Submit actual GPU work
4. Compare the output buffers



GPU Testing

GPU side:

```
f(X) = Y  
X, Y = set of buffers
```

Practically:

- Full render passes with snapshot testing

Future:

- Isolate shaders and capture their output (e.g. OpenGL transform feedback, compute shaders, etc.)



VM Testing



NixOS VM Testing





~~NixOS~~ VM Testing

Full system reproducibility to run software in a very specific environment

- Scenario: Complex state of the world requirements
- Kernel versions, software versions, desktop environments, locales, etc.



NixOS? VM Testing

```
pkgs.testers.runNixOSTest {
  name = "minimal-test";

  nodes.machine = { config, pkgs, ...
}: {
  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    packages = with pkgs; [
      firefox
      tree
    ];
  };
};

testScript = ''

machine.wait_for_unit("default.target")
machine.succeed("su -- Alice -c
'which firefox'")
machine.fail("su -- root -c 'which
firefox'")
'';
}
```

What makes it special:

A full, first party testing framework powered by Nix.

- First party
- Testing Framework
- Powered by Nix



NixOS VM Testing

Step one, defining the machine(s):

```
nodes.machine = { config, pkgs, ... }: {  
  users.users.alice = {  
    isNormalUser = true;  
    extraGroups = [ "wheel" ];  
    packages = with pkgs; [  
      firefox  
      tree  
    ];  
  };  
};
```



NixOS VM Testing

Step two, defining the tests:

```
testScript = ''  
    machine.wait_for_unit("default.target")  
    machine.succeed("su -- Alice -c 'which  
firefox'")  
    machine.fail("su -- root -c 'which  
firefox'")  
'';
```



NixOS VM Testing

Step three, running the tests:

```
# Run all tests  
$ nix flake check
```

```
# Get a Python REPL  
$ nix run .#test-gnome-fcitx5-hangful
```

```
# Build a single test  
$ nix build .#checks.<system>.<name>
```

NixOS VM Testing

In practice:

- Input methods (Japanese, Korean, etc.)
- Desktop integration ("Open in Ghostty" context menu)
- Window decorations (CSD vs. SSD)
- X11 vs. Wayland
- Broken dependencies
- Distro simulation (e.g. exact versions of dependencies from Debian 12)



NixOS VM Testing

Resources:

- https://wiki.nixos.org/wiki/NixOS_VM_tests
- <https://aorith.github.io/posts/nixos-integration-tests/>
- <https://bmcgee.ie/posts/2025/02/nixos-the-power-of-vm-tests/>
- <https://nixcademy.com/posts/nixos-integration-tests/>

Thank You!

Review:

-  Snapshot Testing
-  Keep Context Close
-  Isolate Side Effects
-  GPU Testing
-  NixOS VM Testing

